High Performance Computing

CS5013

Lab 14 – MPI: Sum of N Double Precision Floating Point Numbers

Using Point-to-Point Communication Without Reduction Construct

---

R Abinav — ME23B1004

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int rank, size;
    int N = 16;
    double local_sum = 0.0;
    double total_sum = 0.0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_n = N / size;
    double *local_data = (double *)malloc(local_n * sizeof(double));

    if (rank == 0) {
        double *data = (double *)malloc(N * sizeof(double));

        printf("Initializing %d double precision numbers:\n", N);
        for (int i = 0; i < N; i++) {
            data[i] = (double)(i + 1);
            printf("data[%d] = %.1f\n", i, data[i]);
        }
        printf("\n");

        for (int dest = 1; dest < size; dest++) {
            MPI_Send(&data[dest * local_n], local_n, MPI_DOUBLE,
                     dest, 0, MPI_COMM_WORLD);
        }

        for (int i = 0; i < local_n; i++) {
            local_data[i] = data[i];
        }

        free(data);
    } else {
        MPI_Recv(local_data, local_n, MPI_DOUBLE, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
    }

    for (int i = 0; i < local_n; i++) {
        local_sum += local_data[i];
    }
    printf("Process %d: local_sum = %.6f\n", rank, local_sum);

    if (rank == 0) {
        total_sum = local_sum;

        for (int src = 1; src < size; src++) {
            double recv_sum = 0.0;
            MPI_Recv(&recv_sum, 1, MPI_DOUBLE, src, 1,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            total_sum += recv_sum;
        }

        double expected = (double)(N * (N + 1)) / 2.0;

        printf("\n=== RESULT ===\n");
        printf("Total number of elements : %d\n", N);
        printf("Number of processes      : %d\n", size);
        printf("Computed Sum             : %.6f\n", total_sum);
        printf("Expected Sum (N*(N+1)/2) : %.6f\n", expected);
        printf("Result is %s\n",
               (total_sum == expected) ? "CORRECT" : "INCORRECT");
    } else {
        MPI_Send(&local_sum, 1, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD);
    }

    free(local_data);
    MPI_Finalize();
    return 0;
}
```

Output

```
abinav@Abinavs-MacBook-Air-97 lab-14 % mpicc -o sum_n_p2p sum_n_p2p.c
abinav@Abinavs-MacBook-Air-97 lab-14 % mpirun -np 4 ./sum_n_p2p
Initializing 16 double precision numbers:
data[0] = 1.0
data[1] = 2.0
data[2] = 3.0
data[3] = 4.0
data[4] = 5.0
data[5] = 6.0
data[6] = 7.0
data[7] = 8.0
data[8] = 9.0
data[9] = 10.0
data[10] = 11.0
data[11] = 12.0
data[12] = 13.0
data[13] = 14.0
data[14] = 15.0
data[15] = 16.0

Process 0: local_sum = 10.000000
Process 2: local_sum = 42.000000
Process 1: local_sum = 26.000000
Process 3: local_sum = 58.000000

=== RESULT ===
Total number of elements : 16
Number of processes      : 4
Computed Sum             : 136.000000
Expected Sum (N*(N+1)/2) : 136.000000
```

Figure 1: Program Output – MPI Sum of N Double Precision Numbers